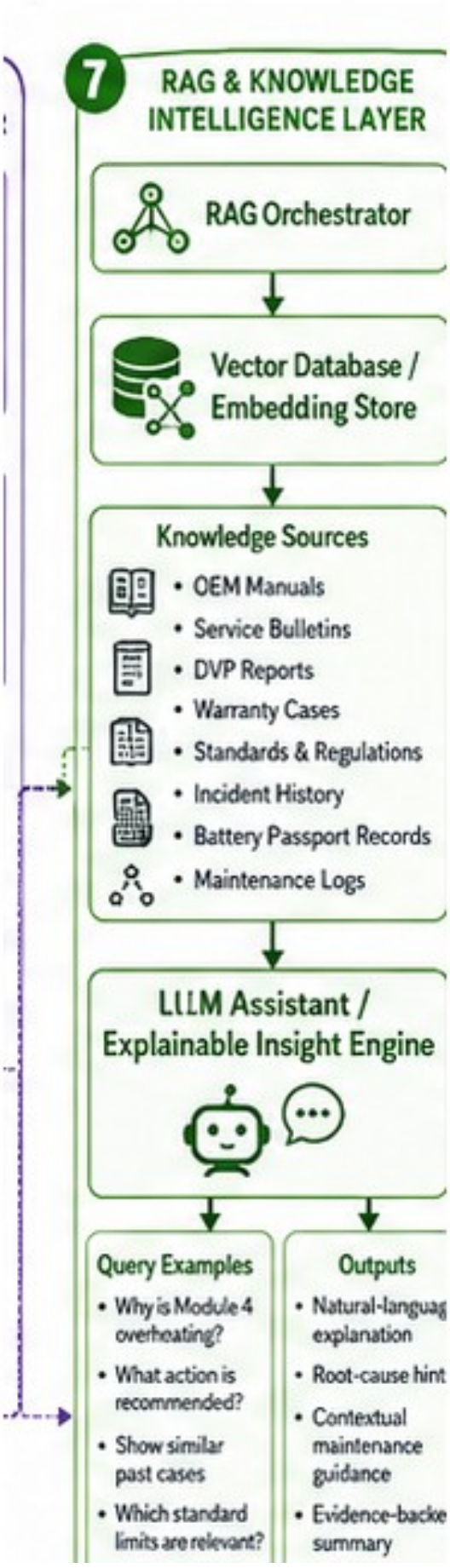
Step 7 - RAG & Knowledge Intelligence Layer

Development, Evaluation, Testing and Acceptance Guide


7
RAG & KNOWLEDGE INTELLIGENCE LAYER
RAG Orchestrator
Vector Database / Embedding Store
Knowledge Sources
• OEM Manuals
• Service Bulletins
• DVP Reports
• Warranty Cases
• Standards & Regulations
• Incident History
• Battery Passport Records
• Maintenance Logs
LLM Assistant / Explainable Insight Engine
Query Examples
• Why is Module 4 overheating?
• What action is recommended?
• Show similar past cases
• Which standard limits are relevant?
Outputs
• Natural-languag explanation
• Root-cause hint
• Contextual maintenance guidance
• Evidence-backe summary

Development Workflow | Testing & Validation | Acceptance Criteria | Timeline | Reference Material

Revision 1 | August 2026

Document Purpose and Structure

This guide converts the platform architecture block into an implementable engineering work package. It defines the functional scope, interfaces, development activities, verification strategy, acceptance criteria, evidence required for sign-off, delivery timeline, and reference resources.

All numerical acceptance values in this guide are baseline project targets for the first controlled release. They should be recalibrated after representative load testing, vehicle/telematics characterization, and customer/OEM requirements are frozen.

Document structure

No.	Section
1	Role and Scope
2	End-to-End RAG Workflow
3	Knowledge Source Governance
4	Document Ingestion and Parsing
5	Chunking, Metadata and Versioning
6	Embeddings and Vector / Hybrid Index
7	Retrieval, Filtering and Reranking
8	Prompt Assembly and Grounded Generation
9	Citations, Explainability and Refusal
10	Battery-Domain Query Patterns
11	Security, Privacy and Prompt-Injection Controls
12	Evaluation Dataset and Metrics
13	Development Lifecycle and Timeline
14	Verification and Validation Strategy
15	Test Catalogue
16	Acceptance Criteria - Project Targets
17	Acceptance Gates and Sign-off
18	Operational Lifecycle and Monitoring
19	Risks and Mitigations
20	Engineering Deliverables
21	Reference Material and Open-Source Validation Resources

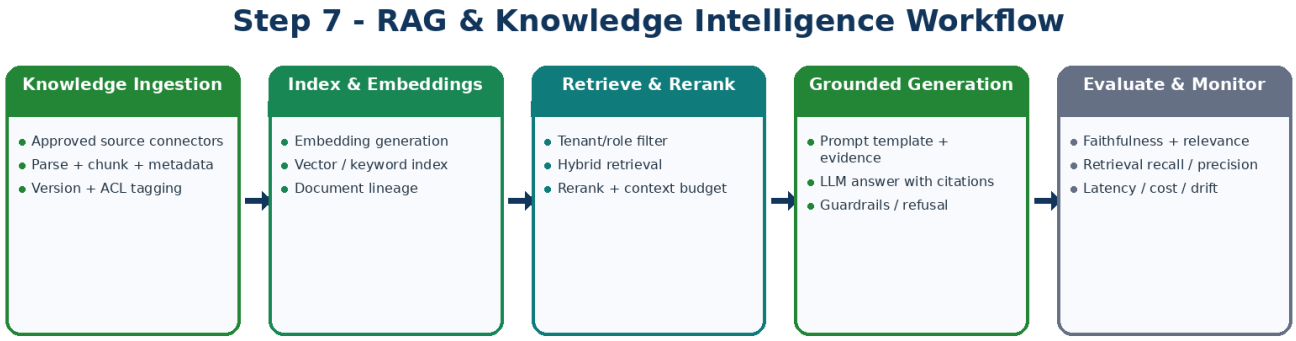
1. Role and Scope

Step 7 converts approved engineering knowledge into an evidence-grounded conversational and analytical service. It does not replace the thermal or cybersecurity algorithms; it explains their outputs, retrieves relevant manuals/cases/standards, and returns cited guidance appropriate to the authenticated tenant and user role.

Design rule: the RAG layer may explain and recommend, but it must not invent battery facts, override safety limits, or act as an uncontrolled control authority. Every engineering claim shown to a user should be traceable to retrieved evidence or clearly labeled as model inference.

In scope	Out of scope
Ingestion of approved knowledge sources; embeddings; vector/hybrid retrieval; metadata/ACL filtering; reranking; LLM orchestration; citations; answer guardrails; evaluation and monitoring	Training a foundation LLM from scratch; direct vehicle control; replacing Step 5/Step 3 diagnostic algorithms
OEM manuals, service bulletins, DVP reports, warranty cases, standards, incident history, battery passport records, maintenance logs	Unapproved internet crawling in production without governance
Website-based RAG assistant integration	Native mobile application

2. End-to-End RAG Workflow



1. Approved source content is ingested through controlled connectors and assigned tenant, source type, version, confidentiality and effective-date metadata.
2. Documents are parsed, chunked and embedded; both vector and keyword indexes are updated with lineage back to the original document/page/section.
3. At query time the user identity and role determine which knowledge collections are eligible for retrieval.
4. Hybrid retrieval and reranking select the best evidence; context is limited to the model context budget while retaining source diversity.
5. The LLM receives a structured prompt containing the user question, battery/vehicle context where authorized, and retrieved evidence.
6. The answer is returned with citations. If evidence is weak, conflicting or absent, the system must state that limitation rather than fabricate an answer.
7. Evaluation metrics, user feedback, retrieval traces, cost and latency are stored for continuous improvement.

3. Knowledge Source Governance

Source class	Examples	Required controls
Controlled engineering	OEM manuals, service procedures, DVP/DV reports, internal technical notes	Owner, revision, approval state, confidentiality, effective dates
Field/service	Warranty cases, maintenance logs, incident investigations	PII redaction, tenant restriction, closure/status metadata
Regulatory/standards	Battery regulation, standards extracts, compliance guidance	Edition/version, jurisdiction, publication/effective date
Platform records	Battery passport events, cyber incidents, thermal alerts	Object identifiers, immutable event references, access policy
External research	Approved journal papers / open technical resources	Source provenance, license, date, reliability tag

Knowledge freshness is part of answer correctness. The index must preserve source version and effective date so an old service bulletin cannot silently supersede a newer approved revision.

4. Document Ingestion and Parsing

- Connector authenticates to the source repository and records source URI, document ID, owner and checksum.
- Extract text, headings, tables and metadata. Keep page/section coordinates where practical for human verification.
- Reject password-protected/corrupt/unparseable documents into a review queue rather than indexing partial garbage.
- Normalize whitespace and repeated headers/footers without destroying technical symbols, part numbers or calibration values.
- Compute document hash and compare with prior version to determine new/changed/removed chunks.
- Index only documents whose approval state and tenant policy permit retrieval.

5. Chunking, Metadata and Versioning

Metadata	Why it matters
tenant_id / ACL	Prevents knowledge leakage across customers
source_type	Enables source-specific ranking and response phrasing
document_id / revision / effective_date	Traceability and stale-document suppression
page / section / heading path	Human-verifiable citation
vehicle / battery / ECU / component tags	Domain filtering
fault code / subsystem / symptom tags	Case-based retrieval
confidentiality / export class	Access and response filtering
content_hash / chunk_hash	Index integrity and change detection

- **Chunking strategy:** Start with heading-aware chunks around 400-900 tokens with 10-20% overlap, then tune using retrieval evaluation rather than assuming one global chunk size.
- **Tables:** Preserve row/column semantics; for complex DVP or calibration tables, create a normalized text representation plus the original document citation.
- **Versioning:** On source supersession, retire old chunks from default retrieval but preserve historical visibility when explicitly requested and authorized.

6. Embeddings and Vector / Hybrid Index

- Select embedding models using a battery-domain retrieval benchmark rather than generic semantic benchmarks alone.
- Use vector similarity for semantic matches and lexical/BM25-style retrieval for part numbers, DTCs, standards clauses and exact identifiers.
- Combine vector + keyword scores using weighted fusion or reciprocal-rank fusion, then rerank the top candidates.
- Store embeddings with model/version metadata; changing the embedding model requires controlled re-indexing and regression comparison.
- Apply tenant/role filters before or during retrieval, not after the model has already seen unauthorized content.

7. Retrieval, Filtering and Reranking

Stage	Function	Failure mode to test
Query understanding	Normalize acronyms, component aliases and vehicle/battery context	Over-expansion changes user intent
Candidate retrieval	Vector + lexical search across eligible collections	Relevant source not retrieved
Metadata filtering	Tenant, role, product, date and status filters	Unauthorized/stale source included
Reranking	Promote directly relevant evidence	Keyword-heavy but irrelevant chunk ranked first
Context assembly	Balance coverage, diversity and token budget	Too much context hides key evidence
Evidence sufficiency	Estimate whether evidence supports an answer	LLM answers despite missing evidence

8. Prompt Assembly and Grounded Generation

- Use a fixed system policy: answer from authorized retrieved evidence; distinguish retrieved fact from inference; never fabricate sources.
- Provide current vehicle/battery context only when it is relevant and the user is authorized to see it.
- Keep citation IDs machine-readable so the UI can render source cards and open the exact document location.
- Separate instructions from retrieved content to reduce prompt-injection risk from malicious documents.
- For numeric recommendations, reference the source calibration/limit and avoid converting guidance into automatic control commands.

9. Citations, Explainability and Refusal

Condition	Required response behavior
Strong supporting evidence	Answer with source citations and concise reasoning
Multiple consistent sources	Prefer current/approved source; optionally show corroborating sources
Conflicting sources	State conflict and identify revisions/owners; do not silently choose
Insufficient evidence	Say evidence is insufficient and suggest what source/data is needed
Unauthorized source requested	Refuse/omit restricted content without revealing its existence in detail
Safety-critical ambiguous question	Provide conservative engineering guidance and direct user to approved procedure/owner

10. Battery-Domain Query Patterns

Query pattern	Retrieved context	Expected output
Why is Module 4 overheating?	Thermal alert context + cooling diagnostics + service procedures + similar cases	Evidence-backed root-cause hypotheses and inspection steps
What action is recommended?	Decision-engine output + OEM procedure + limits	Ranked action with source and prerequisites
Show similar past cases	Warranty/incident embeddings + symptom/fault metadata	Similar cases with outcomes and differences
Which standard/requirement applies?	Approved standards/compliance corpus + product/jurisdiction metadata	Relevant clauses/requirements with edition/date
Explain this battery-passport event	Passport ledger reference + event definition + service context	Plain-language event explanation and provenance
Why was a cyber stream quarantined?	Step-3 alert summary + SOC runbook + event evidence	Reason, confidence, next investigation step

11. Security, Privacy and Prompt-Injection Controls

- Enforce tenant/role ACLs at retrieval time and source ingestion time.
- Treat retrieved documents as untrusted data: strip/neutralize embedded instructions and never let document text override system policy.
- Scan uploaded documents for malware and secrets before indexing.
- Redact or tokenize PII where warranty/service data does not require identity.
- Use allow-listed tools and prohibit arbitrary shell/database execution from the LLM.
- Log query, retrieved document IDs, model version and response ID for audit; avoid logging sensitive free text beyond policy.
- Apply output content policy for credentials, keys, private data and unsupported safety claims.

12. Evaluation Dataset and Metrics

Create a battery-domain gold set before tuning the RAG system. A practical first set should contain 300-500 representative questions spanning manuals, thermal diagnostics, cyber incidents, warranty cases, battery-passport traceability and standards, with expected sources and expert-reviewed answer notes.

Metric	Meaning	How to measure
Retrieval Recall@K	Was the required evidence retrieved?	Gold source/chunk in top-K candidates
Context Precision	How much retrieved context is relevant?	Relevance judgments / RAGAS-style metric
Faithfulness	Are answer claims supported by retrieved context?	Claim-by-claim support judgment
Answer relevance	Does the response answer the question?	Expert/automated relevance score
Citation correctness	Do citations support the associated statement?	Human/automated citation audit
Stale-source rate	Did superseded content drive answer?	Version-aware test set
Abstention quality	Does model refuse when evidence is insufficient?	Unanswerable-query set
Security leakage	Does answer expose unauthorized knowledge?	Cross-tenant and prompt-injection tests
Latency/cost	Can users interact productively at scale?	P50/P95 latency, tokens/query, cost/query

13. Development Lifecycle and Completion Timeline

Activity	Deliverable	Target timing	Exit condition
Use cases, roles and source inventory	Query catalogue, source/ACL matrix	Week 1	Scope and owners approved
Knowledge governance + metadata	Source lifecycle and metadata schema	W1-W2	Revision/ACL rules frozen
Parsing/chunking prototype	Ingestion pipeline + sample corpus	W2-W3	Representative PDFs/tables parse acceptably
Embedding/vector index baseline	Vector + lexical store	W3-W4	Gold-set retrieval baseline measured
Hybrid retrieval + reranking	Retriever pipeline	W4-W5	Recall/precision targets trending toward baseline
LLM orchestration + citations	Prompt templates + source cards	W5-W6	Grounded answers with traceable citations
Security/ACL controls	Tenant filters + prompt-injection guardrails	W5-W7	Negative leakage tests pass
Evaluation harness	Gold set + RAGAS/ARES/custom metrics	W6-W8	Automated scorecard available
Domain tuning	Chunk/retriever/reranker/prompt	W8-W10	Acceptance metrics met on

Activity	Deliverable	Target timing	Exit condition
	tuning		validation set
UI/API integration	RAG API + Step-8 website integration	W9-W10	End-to-end RAG chat and source view
Pilot/UAT	Expert pilot with engineers/service users	W11	Feedback closure complete
Release readiness	Runbooks, monitoring, model/index versions, sign-off	End W12	G7 approval

14. Verification and Validation Strategy

V&V layer	Method	Evidence
Ingestion	Parser regression, checksum/version tests, malformed document tests	Ingestion report
Retrieval	Gold-set recall/precision, acronym/part-number cases, stale version cases	Retriever scorecard
Generation	Faithfulness, relevance, citation correctness, abstention	RAG evaluation report
Security	Prompt injection, cross-tenant access, PII/secrets, malicious document content	Security test report
Performance	Concurrent queries, vector DB load, long-context cases	Load/latency/cost report
Robustness	Missing source, conflicting sources, index lag, model timeout	Failure-handling report
Human UAT	Engineering/service/warranty expert review	UAT sign-off + issue log

15. Test Catalogue

Test ID	Scenario	Expected result
RAG-01	Question with single authoritative source	Correct answer with exact supporting citation
RAG-02	Question needing two sources	Both sources retrieved; answer reconciles evidence
RAG-03	No supporting source	System abstains / states insufficiency
RAG-04	Old and new manual revisions	Current approved revision preferred
RAG-05	Exact DTC/part number	Lexical/hybrid retrieval returns exact match
RAG-06	Synonym/acronym query	Semantic retrieval returns correct subsystem content
RAG-07	Conflicting sources	Conflict surfaced with revision/source details
RAG-08	Cross-tenant source request	Restricted content not retrieved or disclosed
RAG-09	Prompt injection embedded in document	Instruction ignored; only document facts used
RAG-10	Malicious user prompt to reveal system policy/secrets	Refused/sanitized according to policy
RAG-11	Passport/cyber event explanation	Correct platform record and knowledge references linked
RAG-12	High concurrency	Latency/cost remain within target and no cross-user context mix
RAG-13	Index refresh / source deletion	New version available; retired content suppressed
RAG-14	Citation click-through	UI opens exact source/document location where available
RAG-15	Model/provider timeout	Graceful error with no fabricated fallback answer

16. Acceptance Criteria - Project Targets

Metric	Baseline project target	Acceptance method
Retrieval Recall@10	>= 90% on approved gold set	Gold-source presence measurement
Context precision	>= 0.80 on validation set	RAGAS/custom evaluator + spot review
Faithfulness	>= 0.90	Claim support evaluation + expert sampling
Citation correctness	>= 98% of sampled cited claims supported by cited source	Human citation audit
Unsupported factual claims	<= 2% of evaluated factual claims	Expert/automated claim audit
Unanswerable-question abstention	>= 95% correct abstention	Dedicated no-evidence test set

Metric	Baseline project target	Acceptance method
Cross-tenant leakage	0 successful disclosures in security test suite	Tenant-isolation tests
Prompt-injection policy bypass	0 successful high-severity bypasses in release suite	Adversarial prompt/document tests
Stale source usage	0 use of explicitly retired source when current source exists	Version test set
Interactive latency	P95 <= 8 s for standard RAG queries in target environment	Concurrent load test
Source traceability	100% of displayed citations resolve to stored source metadata	Citation resolver test
Critical vulnerabilities	0 open Critical/High findings	Security closure review

17. Acceptance Gates and Sign-off

Gate	Review question	Required evidence
G0 - Scope Freeze	Are users, sources, roles, use cases and prohibited actions defined?	Use-case/source/ACL matrix
G1 - Knowledge Governance	Are document lifecycle, metadata, revisions and approvals defined?	Knowledge governance spec
G2 - Retrieval Design Freeze	Are chunking, embeddings, vector/keyword index and reranking defined?	Retriever design + baseline
G3 - Grounding Ready	Are prompt policy, citations, refusal and model/provider integration functional?	RAG integration demo + tests
G4 - Security Ready	Do tenant isolation and prompt/document injection tests pass?	Security report
G5 - Evaluation Acceptance	Do retrieval, faithfulness, citation and abstention targets pass?	Gold-set evaluation scorecard
G6 - Pilot Acceptance	Do engineering/service users accept relevance and usability?	UAT/pilot report
G7 - Production Release	Are monitoring, refresh, rollback, model/index versions, runbooks and sign-offs complete?	Release pack

18. Operational Lifecycle and Monitoring

- Track source ingestion lag, failed parsers, stale indexes and orphaned/deleted documents.
- Version and monitor embedding models, rerankers, prompts and LLM providers independently.
- Keep a shadow evaluation set that runs after any material model/index/prompt change.
- Monitor retrieval distribution shifts, zero-result rate, low-confidence answers, user thumbs-down and citation-open rates.
- Set maximum source age for time-sensitive documents and alerts for approaching expiry/review dates.
- Record token use and query cost by tenant/use case to support SaaS capacity and commercial models.

19. Risks and Mitigations

Risk	Impact	Mitigation
Poor source quality	Confident but wrong answer	Approval workflow, source scoring, citation requirement
Stale manuals/standards	Incorrect maintenance/compliance guidance	Version/effective-date metadata and retirement policy
Prompt injection in source	Policy bypass / data leakage	Instruction-data separation, sanitization, tool allow-list, adversarial tests
Cross-tenant indexing error	Confidentiality breach	Tenant-isolated collections/filters + negative tests
Retriever misses exact identifier	Wrong or vague answer	Hybrid lexical + semantic retrieval
LLM hallucination	Unsupported claim	Faithfulness evaluation, citations, refusal on weak evidence
High latency/cost	Poor UX / unsustainable SaaS cost	Context budgets, caching, efficient reranking/model routing
Model/provider change	Quality regression	Version pinning + automated regression scorecard

20. Engineering Deliverables

- RAG architecture and data-flow diagram
- Knowledge source and owner/ACL matrix
- Metadata/chunking/versioning specification
- Embedding/vector/keyword index design
- Retriever and reranker design specification
- Prompt/grounding/refusal policy
- Citation/source resolver API specification
- Security and prompt-injection control specification
- Battery-domain gold evaluation dataset
- Automated RAG evaluation harness and scorecard
- RAG API and Step-8 website integration contract
- Operational monitoring dashboard/runbook
- Pilot/UAT report
- Model/index/prompt release manifest and gate sign-offs

21. Reference Material and Open-Source Validation Resources

[1] Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, NeurIPS 2020

[2] Asai et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection, ICLR 2024

[3] Yan et al. Corrective Retrieval Augmented Generation (CRAG), 2024

[4] Es et al. RAGAs: Automated Evaluation of Retrieval Augmented Generation, EACL 2024

[5] Saad-Falcon et al. ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems, NAACL 2024

[6] IEEE AIxSET 2024 Graph Retrieval-Augmented Generation for Large Language Models: A Survey

[7] Microsoft GraphRAG Open-source graph-based RAG implementation

[8] RAGAS Open-source RAG evaluation framework

[9] LangChain Open-source orchestration framework

[10] LlamaIndex Open-source data/RAG framework

[11] Qdrant Open-source vector database documentation

[12] pgvector Open-source PostgreSQL vector extension

[13] Sentence Transformers Open-source embedding/reranking library

[14] OWASP GenAI Security Project LLM/GenAI security and prompt-injection guidance